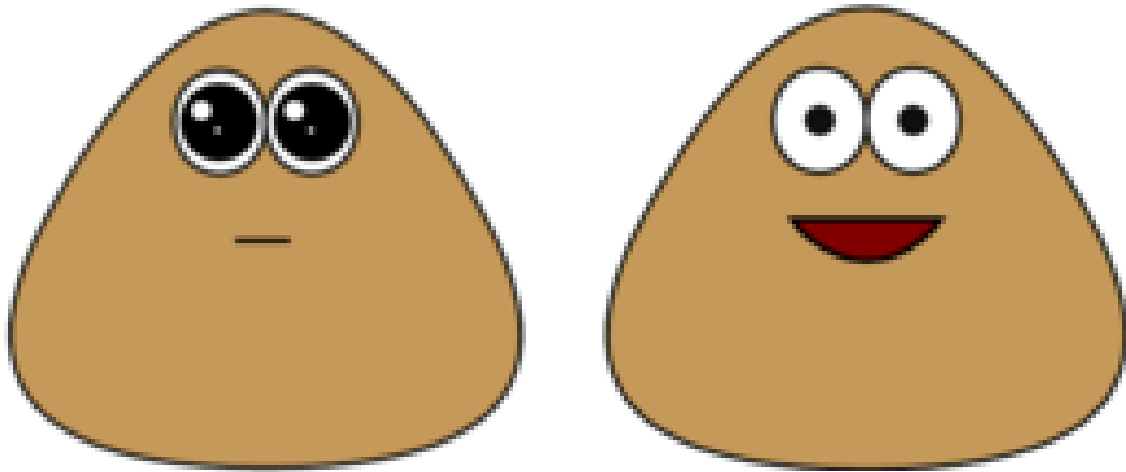


Programmering B Projekt Beskrivelse

Maze-spil

Hanna Vera Rojas og Zulaikha Rahima Jespersen

3.P



Lektor: Rong Fu

Forsøgsdeltager: Hanna Vera Rojas og Zulaikha Rahima Jespersen

Afleveringsfrist: 10/05/2026

Indholdsfortegnelse

Beskrivelse af idé	3
Problemstilling	3
Beskrivelse af programmets funktioner	3
Kravspecifikationer.....	4
Minimumskrav.....	4
Ekstrakrav	4
Tidsplan.....	4
Arbejdsfordeling	5
Faglige fokusmål	5
Hanna	5
Zulaikha	5
Bilag	6
Mindmap.....	6

Beskrivelse af idé

I dette eksamensprojekt vil vi udvikle et slags puzzle-game som har fokus på labyrinter. Vi har taget udgangspunkt i en karakter fra et populært virtuelt kæledyrsspil kaldt Pou som blev udviklet og udgivet af Paul Salameh i 2012.

I vores program vil vi tage denne karakter, Pou, og tvinge den gennem labyrinter. Vi har tænkt os at lave 2 game modes:

Det første game mode kan man i princippet spille i uendelig tid, men i hver labyrint har man 1 minut til at gennemføre banen og hvis man ikke gennemføre den dør man. Hvis man i modsætning når at klare banen får Pou noget mad og vokser som kommer til at tilføje et point. Når man har 10 point, eksploderer Pou og bliver til en ny farve. Igennem banen kan man samle et ur som vil give mere tid.

Det andet game mode kommer til at have 10 baner med stigende sværhedsgrad, hvor målet er at få Pou hjem.

Problemstilling

Hvordan kan vi udvikle et labyrint-spil der ville passe ind det virtuelle kæledyrsspil kaldt Pou med anvendelsen af algoritmer til at danne udfordrende banerne?

Beskrivelse af programmets funktioner

Spillet kommer til at genere labyrinter af forskellige sværhedsgrader ved brug af tværgående algoritmen DFS (depth first search). Der vil også være en tidsfunktion (timer) som registrerer hvor meget tid det tager for spilleren at løse labyrinten.

Spilleren kommer spillet til at tage input til at bevæge sig med WASD og piletasterne. Der vil være en tilføjelse af kollision med objekter som et tidssur der giver mere tid og mad i slutningen af banen der giver point til ens highscore og sender en videre til næste bane. Når disse kollisioner kunne vi godt tænke os at tilføje lydeffekter.

Kravspezifikationer

For at sikre at vi kommer i mål med vores produkt vil vi lave målbare krav som kan hjælpe os med at vurdere om vi har opnået den ønskede kvalitet.

Minimumskrav

1. Kan danne labyrinter med 3 forskellige sværhedsgrader.
2. En karakter der kan bevæge sig gennem labyrinten.
3. En tidsbegrænser.
4. Point system.

Ekstrakrav

1. Implementering af UI der matcher Pou spillet samt lyde.
2. Menu med infinity mode (gamemode 1), story (gamemode 2) controls, og quit.
3. Få Pou til at eksplodere efter at få 10 point og skifte farve.
4. Lav game mode 2 (storymode).

Tidsplan

For at lave en tidsplan udformede vi et [Gantttdiagram.xlsx](#) med forskellige formål. Disse formål består af:

1. Generer og vis statisk labyrint i konsollen (grundlæggende GUI)
2. Spilleren bevæger sig i labyrint-GUI, kollisioner.
3. Løs labyrinten (næste niveau (større))
4. Endeløse labyrinter, høje scores.
5. Full game: Menus, bug fix, osv.
6. Færdiggørelse af minimums Krav
7. Ekstra krav
8. Individuel synopsis

Ud fra disse formål har vi lavet en kalender der viser, hvornår vi kommer til at udføre dem.

Arbejdsfordeling

I forhold til arbejdsfordeling tager vi udgangspunkt i MVC og Class diagram.

MVC (Model, View, Controller)	
Model	Hanna
View	Zulaikha
Controller	Zulaikha
Class	
Maze	Hanna
Player	Zulaikha
Point	Zulaikha
Menu	Zulaikha
Pause menu	Zulaikha

Faglige fokusmål

Gennem projektets forløb kommer vi individuelt til at fokusere på forskellige faglige fokusområder.

Hanna

Hanna har tænkt sig at fokusere på algoritmerne bag vores dannelse af banerne.

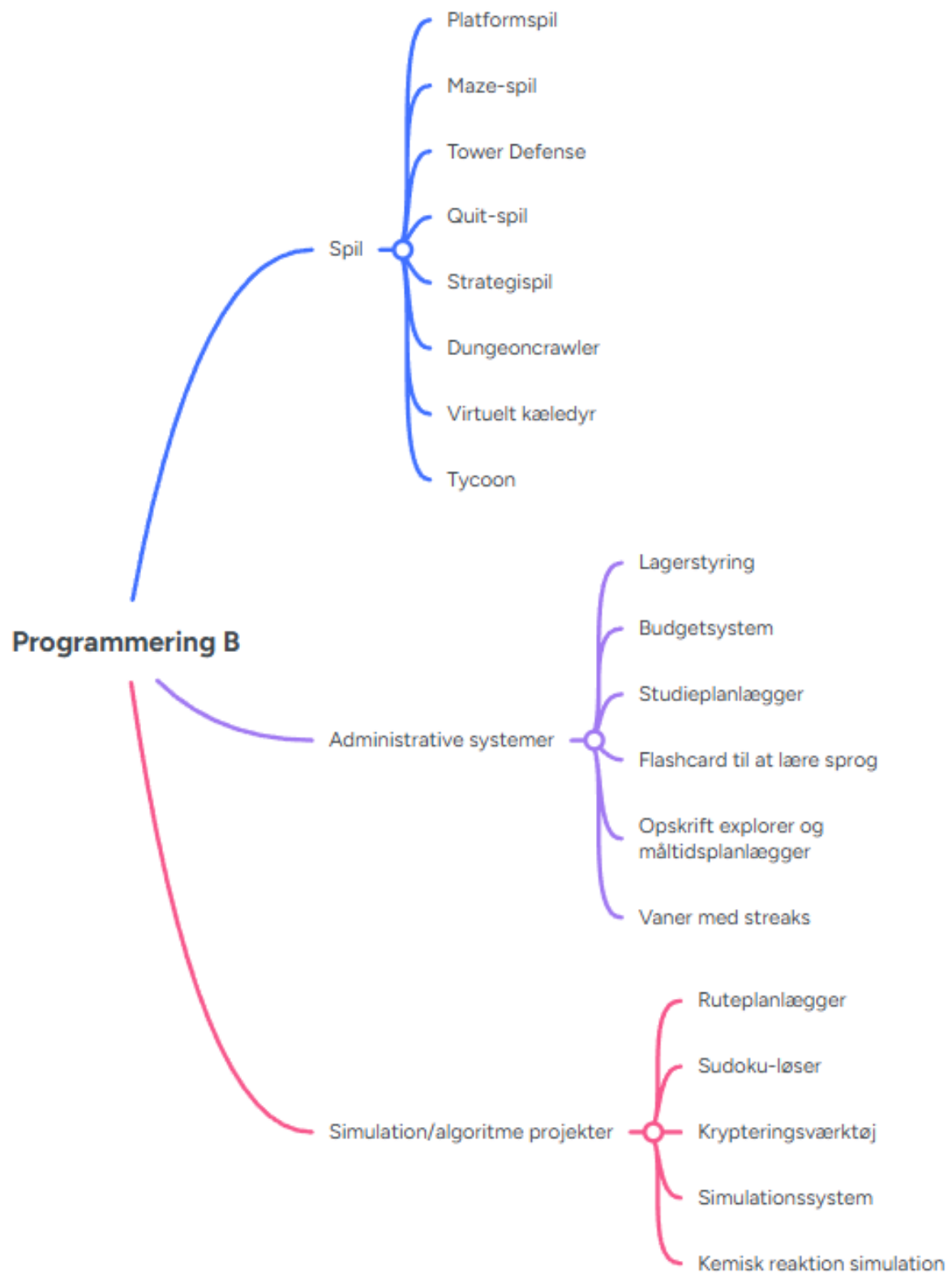
Zulaikha

Zulaikha har tænkt sig at fokusere på graphical user interface (GUI) bag vores produkt samt de sprites og assets vi kommer til at anvende.

Vi kommer begge to til at arbejde på game engine samt lave test og dokumentation af vores forskellige faglige fokusområder.

Bilag

Mindmap



Algoritmer

Programmet kommer til at indeholde flere funktioner. For det første kommer til at anvende DFS (depth first search) algoritmen til at generere labyrinter i n-dimension. DFS behandler en labyrint som et træ hvor grenene er vejene igennem labyrinten. Til at generere labyrinten bygger DFS labyrinters 'træ' på en tilfældig måde indtil den er fuldført. DFS er en traverseringsalgoritme (traversal algorithm). Disse typer algoritmer går igennem 'træ' grafer dvs. grafer der starter i kernen og forgrener sig fra den. (Brilliant.org, u.d.) I denne kategori har vi DFS (depth first search) og BFS (breadth first search) og begge karakteriserer sig for at besøge hver træknude en enkelt gang, forskellen ligger i hvilket rækkefølge træknuderne behandles. (Wikipedia, u.d.)

Et 2D labyrint er en rækkefølge af stiger med vægge der adskiller dem. Til at gøre det enkelt kan man se labyrinten som et 2D gitter med en højde og en bredde og celler der skaber (x,y) koordinater. Hvis vi skal bruge det til at skabe en labyrint skal vi tænke på at hvert punkt er ligesom en træknude og hver træknude har muligheden for at forgrene sig til dens 4 naboceller (bortset for kant og hjørne celler som har mindre forgrenings muligheder).

Hvad gør vores algoritme:

1. Generere en gitte med firkantet celler adskilt af vægge.
2. Vælger en celle (Node) N tilfældigt
3. Skubber N i en kø Q
4. Sætter N som besøgt
5. Vælger tilfældigt en felt A der ligger ved siden af N som ikke er markeret som besøgt. Hvis alle A der ligger ved siden af N har været besøgt:
 - a. Fortsæt med at gå igennem køen af N som er blevet besøgt, indtil den finder en N, som har en u besøgt A felt. Sæt den her felt som N og gå til step 4.
 - b. Hvis Q.length == 0 dvs. der er ikke en u besøgt N: stop.
6. Bryde væggen mellem N og A
7. Sæt A til at være N
8. Tilbage til step 2

(Cope, 2022)

Hvordan bestemmer vi sværheden?

Størrelsen af Q efter den første omgang af algoritmen er blevet kørt (steps 1-7) siger noget om sværheden af labyrinten da den viser afstanden mellem start og slut. Men dette er ikke altid rigtigt da opbygning o den samlede længde af stien fra start til slut er hvad der bestemmer labyrinters sværhedsgrad.

Vi kan også evaluere den tidskompleksitet, det ville tage en algoritme at løse vores labyrinth, for at danne et overblik af laberyntens sværdhedsgraden. Hvis vi forestiller os at hver celle i labyrinthen er en node og hver celle der omringer den som kan blive udforsket er en kant. Så er antallet af nodes (V) og antallet af kanter (E). dermed kan tidskompleksitet måles ved at sige at $O = (|V| + |E|)$. Samtidig kan rumkompleksitet af labyrinthen bestemmes ved $O = |V|$ hvilket også betyder at vores call stack kommer i den værste tilfælde til at have størrelsen $O = |V|$.

Til at generere vores maze har vi brugt:

<https://github.com/tonypdavis/PythonMazeGenerator>